

Sistema de Pasajeros

El sistema de pasajeros de Announce tendrá por objetivo evaluar y puntuar la experiencia de los pasajeros.

Partiendo de la actual generación de pasajeros simulados que se embarcan al comienzo de cada vuelo, gestionaremos cuatro atributos para evaluar la experiencia.

1. Los 4 atributos — definición y rango

Todos en escala **0-100**, donde 100 = mejor estado posible (para evitar confusión de "¿más alto es peor o mejor?" en el código y en la UI).

Atributo	100 significa	0 significa
Saciedad (ex-"hambre")	Satisfecho	Hambriento
Confort fisiológico (ex-"baño")	Cómodo	Urgencia
Calma (ex-"miedo")	Tranquilo	Aterrorizado
Entretenimiento (ex-"aburrimiento")	Entretenido	Muy aburrido

Vamos a invertir la semántica de hambre/baño/aburrimiento a "positivo = 100" (en vez de pensar en hambre/miedo como algo que sube). Es un detalle chico pero evita bugs de signo invertido y hace que el score final sea directamente interpretable (más alto siempre es mejor, en los 4 atributos y en el agregado).

2. Valores al abordar

- **Saciedad:** random entre 80-100 (recién comieron o están por comer, no hay motivo para que arranque bajo)
- **Confort fisiológico:** random entre 85-100 (tu regla original, $\geq 70\%$ en la métrica invertida = ≥ 70 acá también, pero le subiría un poco el piso a 85, porque en la métrica original "ganas de ir al baño $> 70\%$ " ya es bastante incómodo para arrancar un vuelo)
- **Calma:** random entre 40-100, **pero modulado por arquetipo** (el arquetipo "miedo a volar" debería arrancar en un rango más bajo, tipo 20-50, mientras que "viajero frecuente" arranca 70-100)
- **Entretenimiento:** random entre 60-100

3. Arquetipos como multiplicadores (no lógica separada)

Cada arquetipo define un **set de 4 multiplicadores de deterioro** (qué tan rápido se le cae cada atributo) y un **set de 4 multiplicadores de recuperación** (qué tan bien responde a las acciones del jugador). Ejemplo de la tabla (valores ilustrativos, hay que calibrar):

Arquetipo	Deterioro Calma	Recuperación Calma	Deterioro Entretenimiento
Miedo a volar	x2.0 (se asusta más rápido)	x1.5 (pero también responde mejor a que lo tranquilicen)	x1.0
Ejecutivo apurado	x0.5	x0.5	x1.5 (se aburre rápido, quiere llegar)
Familia con chicos	x1.2	x1.0	x2.0 (los chicos se aburren rápido)
Viajero frecuente	x0.7	x0.8	x0.6

Con 10 arquetipos x 4 atributos x 2 multiplicadores, es una tabla de 80 números — manejable como config/JSON, no como código hardcodeado, así después podemos tunear sin recompilar.

4. Deterioro por tiempo

Cada atributo deberá deteriorarse con el paso del tiempo.

Tasa base por minuto de vuelo, **distinta por atributo** (no todos decaen igual):

- **Saciedad:** decae lineal y constante, ej. -0.15/min (se agota en ~11 horas de vuelo sin comer)
- **Confort fisiológico:** decae lineal, algo más rápido, ej. -0.25/min
- **Calma:** **no decae solo por tiempo en vuelo tranquilo** (o decae casi nada, ej. -0.02/min) — se mueve principalmente por eventos.
- **Entretenimiento:** decae lineal, ej. -0.2/min, pero con un piso — nadie se aburre infinitamente, así que podría estabilizarse alrededor de 20-30 si no hay ninguna intervención

Fase de vuelo también modula esto: en climb/descent el confort fisiológico y la calma decaen más rápido (turbulencia más probable, cinturones puestos = más incomodidad), en cruce estable todo decae más lento.

5. Acciones del jugador (las que ya tenemos + cómo impactan)

Dado que ya tenemos el sistema de anuncios de cabina andando, la conexión más natural para el MVP 1 es que **cada anuncio generado tenga un "tipo" que mapea a qué atributos mejora**. A continuación las tablas de ejemplos:

Tipo de anuncio	Atributo que mejora	Magnitud sugerida
Anuncio de progreso/tranquilizador	Calma	+5 a +15 (según arquetipo)
Anuncio de servicio (bebidas/comida próxima)	Saciedad	+5 a +10
Anuncio informativo/entretenido (landmarks, curiosidades)	Entretenimiento	+5 a +10
Anuncio de aterrizaje/preparación	Confort fisiológico	+10 (saben que ya pueden ir al baño pronto)

Esto le da un propósito de gameplay directo al sistema de anuncios que ya construimos — no es un sistema aislado, se retroalimentan.

6. Eventos negativos (conectados a telemetría real)

Evento (detectado por telemetría)	Atributo afectado	Magnitud
Turbulencia (cambios bruscos en vertical speed / G-force)	Calma	-10 a -25 según intensidad
Viraje o maniobra brusca (bank angle alto y rápido)	Calma	-5 a -15
Aterrizaje duro (vertical speed alto al touch down). Nota: esto todavía no lo estamos calculando.	Calma	-15 a -30
Vuelo prolongado sin ningún anuncio (>X minutos)	Entretenimiento	-5 (penalización por "silencio de cabina")

7. Score final

En vez del promedio plano simple, para el MVP 1 propondría algo un poco más rico pero todavía simple de implementar:

`score_pasajero = promedio(los 4 atributos al momento del aterrizaje)`

`score_vuelo = promedio(score_pasajero de todos los pasajeros trackeados)`

`- penalización_por_varianza`

Donde **penalización_por_varianza** es simple: si el desvío estándar entre pasajeros es alto (algunos muy felices, otros muy mal), restás un poco del score final. No hace falta una fórmula estadística sofisticada — con algo como "si más del 20% de los pasajeros tienen un score individual <40, restar 5-10 puntos al score general" ya cumple el objetivo de no esconder los casos malos detrás de un promedio lindo.

8. Indicador visible durante el vuelo (basado en el mockup actual)

Ya que lo tenemos en el mockup, sugeriría mostrar los 4 atributos como **promedio agregado de todos los pasajeros trackeados**, no por pasajero individual, y con un semáforo simple (verde/amarillo/rojo) además del número — así el jugador capta de un vistazo si algo anda mal sin tener que interpretar "62 es bueno o malo?".

9. Tablas de Incidencia de Puntos

Las tablas de esta sección son de ejemplo, pero los valores reales se tomarán del campo `events.effects` (ver mas adelante el formato con el que el desktop recibirá este dato)

Boarding

Anuncio	Atributo(s)	Magnitud	Nota
Inicio en breve	Entretenimiento	+2	Rompe la espera muerta en terminal
Abordaje iniciado	Entretenimiento	+2	Ídem
Bienvenida de la Tripulación	Calma	+3	Primer contacto humano, efecto chico pero real
Información básica de cabina	Calma	+3	Transmite orden/profesionalismo
Bienvenida del Capitán	Calma	+5	Primera vez que "una autoridad" se dirige a ellos — pesa más
Información Operativa del Vuelo	Calma +5, Entretenimiento +3		Info concreta reduce incertidumbre (calma) y da contenido (entretenimiento)
Cabina rutinario	Entretenimiento	+1	Efecto mínimo, es ruido ambiente

Delays (importante: estos son mitigadores de daño, no mejoras puras)

Anuncio	Atributo(s)	Magnitud	Nota
Demora en la salida	Calma	+8 (relativo)	No mejora vs. estado neutral, sino que reduce la caída que produciría la demora sin explicación. Ver nota abajo
Demora en la salida (Parked)	Calma	+8 (relativo)	Ídem
Demora en Taxi	Calma	+6 (relativo)	Ídem
Demora en Takeoff	Calma	+6 (relativo)	Ídem
Demoras en plataforma (llegada)	Calma	+5 (relativo)	Ídem

Nota de diseño importante acá: las demoras en sí mismas deberían generar un **evento negativo pasivo** (Calma -X, Entretenimiento -X mientras dura la demora sin explicar), y el anuncio correspondiente **mitiga esa caída** en vez de sumar puntos "gratis". Esto es más realista (nadie se pone contento porque le avisan de una demora, pero se frustra mucho menos si se lo explican bien) y evita el incentivo perverso de "provocar demoras para poder anunciarlas y ganar puntos".

Taxi (pre-despegue)

Anuncio	Atributo(s)	Magnitud	Nota
Armar toboganes	—	0	Orden interna, sin impacto directo
Atenuar luces capitán	—	0	Orden interna, sin impacto directo
Demostración de seguridad	Calma	+5	Ritual conocido que transmite profesionalismo/normalidad
Atenuar luces tripulación	Entretenimiento	+1	Señal ambiental de "ya estamos por salir"
Preparar despegue	—	0	Orden interna

Cruise

Anuncio	Atributo(s)	Magnitud	Nota
Servicio a bordo (anuncio)	Saciedad	+3	Anticipación, no el servicio en sí
Servicio comidas	Saciedad	+15	Este es el que más debería pesar en Saciedad — es la acción real, no el aviso
Info general	Entretenimiento +8, Calma +3		Tu anuncio de "progreso de vuelo" que armamos al principio de la conversación — encaja perfecto acá
Duty Free	Entretenimiento	+2	Efecto chico, es más comercial que experiencial
Segundo servicio	Saciedad +5, Confort fisiológico +3		Recolección de bandejas también libera espacio/mesa = confort
Tercer servicio	Saciedad	+8	Similar al primero pero de menor magnitud (vuelos largos, ya vienen mejor)
Migraciones	Entretenimiento	+2	Da algo para hacer (completar formularios)
Anuncio especial (VIP)	Entretenimiento	+5	Efecto de "algo interesante está pasando"

Transversal (turbulencia)

Anuncio	Atributo(s)	Magnitud	Nota
Cinturones capitán (señal)	Calma	+4 (relativo)	Mitiga la caída que ya provocó el evento de turbulencia real detectado por telemetría — no es una mejora neta
Cinturones tripulación (refuerzo)	Calma	+2 (relativo)	Ídem, mitigación adicional

Descent

Anuncio	Atributo(s)	Magnitud	Nota
Top of Descent	Calma +2, Entretenimiento +2		Anticipación de que el vuelo está por terminar
Preparación cabina	Confort fisiológico	+5	Última chance de acomodarse antes del aterrizaje
10.000 pies	Calma	+2	Señal de progreso hacia el final
Info destino	Entretenimiento +5, Calma +3		Info concreta similar al anuncio operativo inicial
Aterrizaje inminente	Calma	+5	Prepara mentalmente, reduce sorpresa
Ocupar lugares	—	0	Orden interna

Taxi to Gate / At Gate

Anuncio	Atributo(s)	Magnitud	Nota
Bienvenida destino	Entretenimiento	+3	Cierre ceremonial positivo
Permanecer sentado	Confort fisiológico	-2	Es una restricción, genera algo de impaciencia (leve, esperado)
Demoras en plataforma	Calma	+5 (relativo)	Ya cubierto arriba en la sección Delays
Desarmar toboganes	—	0	Orden interna
Despedida	Calma +3, Entretenimiento +3		Cierre general positivo del vuelo

Evento Especial en Cabina

Este es distinto a todos los demás — es contenido libre ingresado por el jugador, no tiene un propósito fijo predefinido. Por ahora lo configuraremos sin ningun efecto sobre los pasajeros.

10. Nuevo atributo en tabla Events

Vamos a agregar un nuevo atributo en la tabla events para reflejar cómo cada evento afecta (si es que lo hace) en alguno de los atributos de los pasajeros.

Llamaremos a este campo **'effects'**. Este campo no solo definirá "qué atributo y cuánto", sino también **el tipo de efecto** (mejora neta vs. mitigación de evento) que identificamos como un patrón real:

```
{
  "effects": [
    { "attribute": "calma", "amount": 5, "mode": "boost" },
    { "attribute": "entretenimiento", "amount": 3, "mode": "boost" }
  ]
}
```

Para los casos de mitigación (demoras, turbulencia):

```
{
  "effects": [
    { "attribute": "calma", "amount": 8, "mode": "mitigate" }
  ]
}
```

Y para los que no afectan nada (órdenes internas):

```
{ "effects": [] }
```

El campo **mode** es la parte que más conviene dejar explícita desde el día uno, aunque el MVP 1 solo implemente **boost** — porque agregar **mitigate** después como tipo nuevo en el JSON es gratis, pero si arrancamos sin esa distinción y la necesitamos después, vamos a tener que migrar todos los registros existentes para agregarla.

11. Entidades del Modelo

Entidades del modelo

1. Atributos — tipo base reusable

typescript

```
type AttributeKey = 'saciedad' | 'confortFisiologico' | 'calma' | 'entretenimiento';

interface AttributeState {
  saciedad: number;
  confortFisiologico: number;
  calma: number;
  entretenimiento: number;
}
```

2. Arquetipo — definición estática (config, no cambia durante el vuelo)

typescript

```
interface ArchetypeDefinition {
  id: string;
  name: string;
  deteriorationMultipliers: AttributeState; // multiplica la tasa base de deterioro por atributo
  recoveryMultipliers: AttributeState; // multiplica la magnitud de boost/mitigate por atributo
  boardingRanges?: Partial<Record<AttributeKey, [number, number]>>; // override del rango random al abordar (ej. "miedo a volar" → calma: [20, 50])
}
```

3. Pasajero — instancia runtime (los 10 trackeados)

typescript

```
interface Passenger {  
  id: string;  
  archetypeId: string;  
  seatLabel?: string; // flavor para UI, opcional  
  attributes: AttributeState;  
}
```

Para el MVP 1, como definimos que solo trackeamos 10 y el resto del avión no se simula, no hace falta un flag `isTracked` — todo objeto `Passenger` que exista en memoria es, por definición, uno de los 10.

4. Efecto — la pieza reutilizable que sale del JSONB que armaste

typescript

```
interface PassengerEffect {  
  attribute: AttributeKey;  
  amount: number;  
  mode: 'boost' | 'mitigate';  
}
```

Esta misma interfaz sirve tanto para deserializar el JSONB de los anuncios como para los efectos de eventos de telemetría (turbulencia, etc.) — un solo tipo, dos orígenes distintos de datos.

5. Evento negativo activo — bookkeeping necesario para que **mitigate** funcione

Este es el que probablemente no habías pensado todavía, pero lo necesitas: para que un anuncio "mitigue" un evento (en vez de sumar puntos de la nada), el motor necesita saber **que hay un evento negativo reciente sin mitigar**.

typescript

```
interface ActiveNegativeEvent {  
  id: string;  
  attribute: AttributeKey;  
  triggeredAtSecond: number; // segundo del vuelo en que ocurrió  
  mitigationWindowSeconds: number; // cuánto tiempo sigue siendo "mitigable" (ej.  
90-120seg)  
  mitigated: boolean;  
}
```

Lógica implícita (para cuando pasemos al motor de tick): cuando telemetría dispara un evento negativo (turbulencia), se aplica el impacto negativo inmediato a los pasajeros Y se crea un **ActiveNegativeEvent**. Si dentro de la ventana de mitigación se reproduce un anuncio con efecto **mitigate** sobre el mismo atributo, se consume ese evento (**mitigated: true**) y se aplica la recuperación. Si la ventana expira sin que se reproduzca el anuncio correspondiente, el evento simplemente queda "cobrado" sin mitigar — el jugador perdió la oportunidad.

6. Estado runtime del motor — el estado completo en memoria durante el vuelo

typescript

```
interface PassengerEngineState {  
  passengers: Passenger[]; // 10 elementos  
  activeNegativeEvents: ActiveNegativeEvent[]; // eventos pendientes de mitigar  
  elapsedSeconds: number;  
}
```

7. Resumen final — lo único que se persiste

typescript

```
interface FlightPassengerSummary {  
    flightId: string;  
  
    globalAttributeAverages: AttributeState; // promedio de los 4 atributos entre los 10  
    pasajeros, al aterrizar  
  
    overallScore: number; // el score final (promedio - penalización por varianza)  
  
    variancePenaltyApplied: number;  
}
```

Este es el único objeto que cruza la frontera de "en memoria" a "persistido" — todo lo demás (*PassengerEngineState*, los *Passenger* individuales) se descarta al finalizar el vuelo, tal como planteaste.